Rapport de Sprint Drupal

Développement Avancé de Modules

Entités, Hooks, Configuration, Cache & Migration

Sadiki Abdelkarim

Durée du Sprint : 5 Jours

16 mars 2026

Table des matières

1 Travailler avec les Données (API Entité & Champ, Entités de Configuration) 3

1.1 Contrainte de politique de mot de passe via `hook_entity_base_field_info_alter` 3

1.1.1 Installation . . . 3

1.1.2 Implémentation de la contrainte . . . 3

1.2 Qu'est-ce qu'`AccessResult` et comment fonctionne-t-il ? . . . 5

1.2.1 États principaux . . . 5

1.2.2 Métadonnées de cache . . . 5

1.2.3 Fonctionnement de la vérification d'accès . . . 6

1.3 Générer une entité personnalisée . . . 6

1.4 Récupérer une définition de champ par le code . . . 6

1.5 Plusieurs formateurs de champ pour un même type de champ . . . 7

1.6 Récupérer la configuration d'un module via Drush . . . 8

2 Travailler avec les Hooks 9

2.1 Ajouter un champ de base via `hook_entity_base_field_info` . . . 9

2.2 Rôle de `hook_update_N` . . . 10

2.3 Rôle de `hook_install` . . . 10

2.4 Préfixer les nœuds Article avec `HEY-` via `hook_ENTITY_TYPE_presave` . . . 11

2.5 Rôle de `$entity->original` . . . 11

2.6 Surcharger un hook de thème d'un autre module . . . 12

2.7 Ajouter des suggestions de thème pour l'utilisateur selon le mode d'affichage 13

3 Travailler avec la Configuration & les Fonctionnalités 14

3.1 Configuration de l'environnement . . . 14

3.2 Workflow de synchronisation de configuration . . . 14

3.2.1 Dev : Configuration . . . 14

3.2.2 Production : Import . . . 14

3.2.3 Utilisation de Config Split . . . 15

3.3 Workflow du module Features . . . 15

4 Travailler avec le Cache 17

4.1 Point de terminaison personnalisé `/api/articles` . . . 17

4.1.1 Définition de la route . . . 17

4.1.2 Contrôleur avec mise en cache Max-Age . . . 17

4.1.3 Contrôleur avec tags de cache (Recommandé) . . . 18

4.2 Inspecter les tags de cache via les en-têtes de réponse . . . 19

5 Travailler avec la Migration de Données **20**
5.1 Rôle de `migration_lookup` 20
5.2 Importer depuis une source de base de données MySQL 20
5.3 Annuler une migration 21
5.4 Traiter les données de champ avant l'import 22

Jour 1: Travailler avec les Données (API Entité & Champ, Entités de Configuration)

Référence : https://drupalize.me/guide/work-data-modules

1.1 Contrainte de politique de mot de passe via `hook_entity_base_field_inf`

Pour ajouter une contrainte de mot de passe personnalisée à l'entité utilisateur Drupal, nous installons et configurons d'abord le module **Password Policy**, puis nous implémentons `hook_entity_base_field_info_alter` dans un module personnalisé.

1.1.1 Installation

```
composer require drupal/password_policy
drush en password_policy -y
drush cr
```

Listing 1.1 – Installer Password Policy via Composer

Après avoir activé le module, accédez à `/admin/config/security/password-policy` pour configurer vos règles de politique (longueur minimale, caractères spéciaux, etc.).

1.1.2 Implémentation de la contrainte

Fichier : `mymodule/src/Plugin/Validation/Constraint/PasswordPolicyConstraint.php`

```
<?php

namespace Drupal\mymodule\Plugin\Validation\Constraint;

use Symfony\Component\Validator\Constraint;

/**
 * @Constraint(
 *   id = "PasswordPolicy",
 *   label = @Translation("Contrainte de politique de mot de
    passe"),
 * )
 */
class PasswordPolicyConstraint extends Constraint {
  public $message = 'Le mot de passe ne respecte pas les
     exigences de la politique.';
```

```
}
```

Listing 1.2 – Plugin PasswordPolicyConstraint

Fichier : `mymodule/src/Plugin/Validation/Constraint/PasswordPolicyConstraintValidator.`

```php
<?php

namespace Drupal\mymodule\Plugin\Validation\Constraint;

use Symfony\Component\Validator\Constraint;
use Symfony\Component\Validator\ConstraintValidator;
use Symfony\Component\DependencyInjection\ContainerInterface;
use
   Drupal\Core\DependencyInjection\ContainerInjectionInterface;
use Drupal\password_policy\PasswordPolicyValidator;

class PasswordPolicyConstraintValidator extends
   ConstraintValidator
  implements ContainerInjectionInterface {

  protected $passwordPolicyValidator;

  public function __construct(PasswordPolicyValidator
     $validator) {
    $this->passwordPolicyValidator = $validator;
  }

  public static function create(ContainerInterface $container)
     {
    return new static(
      $container->get('password_policy.validator')
    );
  }

  public function validate($value, Constraint $constraint) {
    if (!$this->passwordPolicyValidator->validatePassword(
          $value, \Drupal::currentUser())) {
      $this->context->addViolation($constraint->message);
    }
  }
}
```

Listing 1.3 – Validateur de contrainte avec injection de service

Fichier : `mymodule.module`

```php
<?php

use Drupal\Core\Entity\EntityTypeInterface;

/**
 * Implements hook_entity_base_field_info_alter().
```

```
 */
function mymodule_entity_base_field_info_alter(
    &$fields, EntityTypeInterface $entity_type) {
  if ($entity_type->id() === 'user'
      && isset($fields['pass'])) {
    $fields['pass']->addConstraint('PasswordPolicy', []);
  }
}
```

Listing 1.4 – hook_entity_base_field_info_alter

1.2 Qu'est-ce qu'`AccessResult` et comment fonctionne-t-il ?

`AccessResult` est l'API orientée objet de Drupal pour exprimer les décisions de contrôle d'accès. Au lieu de retourner un simple booléen, il retourne un objet riche qui contient des métadonnées de cache ainsi que la décision.

1.2.1 États principaux

- **`AccessResult::allowed()`** — L'accès est explicitement accordé.
- **`AccessResult::forbidden()`** — L'accès est explicitement refusé.
- **`AccessResult::neutral()`** — Pas d'opinion ; délègue aux autres vérifications d'accès.

1.2.2 Métadonnées de cache

Les résultats d'accès peuvent contenir des tags de cache, des contextes et un max-age afin que Drupal puisse correctement varier et invalider les réponses en cache :

```
<?php

use Drupal\Core\Access\AccessResult;
use Drupal\Core\Session\AccountInterface;

function mymodule_node_access(
    \Drupal\node\NodeInterface $node,
    $op,
    AccountInterface $account) {

  if ($op === 'view' && $node->bundle() === 'secret') {
    return AccessResult::forbiddenIf(
      !$account->hasPermission('view secret content')
    )
    ->cachePerPermissions()   // Varier le cache par
       permissions
    ->addCacheableDependency($node); // Invalider lors du
       changement du noeud
  }

  return AccessResult::neutral();
}
```

Listing 1.5 – AccessResult avec métadonnées de cache

1.2.3 Fonctionnement de la vérification d'accès

1. Drupal collecte les objets `AccessResult` de tous les hooks et services pertinents.
2. Un seul `forbidden()` annule tous les résultats `allowed()`.
3. Les résultats `neutral()` sont ignorés sauf si tous les résultats sont neutres (par défaut : refusé).
4. Les métadonnées de cache de tous les résultats sont fusionnées pour un cache HTTP correct.

1.3 Générer une entité personnalisée

Créer une entité personnalisée manuellement implique des dizaines de fichiers. L'approche recommandée est d'utiliser **Drupal Console** ou **Drush** avec la génération de code :

```
# G n rer une entit  de contenu
drupal generate:entity:content

# Invites interactives :
# - Nom du module
# - Nom de la classe d'entit
# - Ajouter des op rations CRUD
# - Ajouter un bundle

# Ou avec Drush + Drush Generate :
drush generate entity
```

Listing 1.6 – Générer une entité avec Drupal Console

Le générateur crée le scaffolding suivant :

— `src/Entity/MyEntity.php` — Classe d'entité avec annotations
— `src/Entity/MyEntityInterface.php` — Interface
— `src/MyEntityListBuilder.php` — Liste d'administration
— `src/Form/MyEntityForm.php` — Formulaire d'ajout/édition
— `src/Form/MyEntityDeleteForm.php` — Formulaire de suppression
— `config/schema/mymodule.schema.yml`
— Définitions de routage, liens et permissions

1.4 Récupérer une définition de champ par le code

```php
<?php

// Obtenir toutes les d finitions de champs de base pour un
   type d'entit
$field_definitions = \Drupal::service('entity_field.manager')
  ->getBaseFieldDefinitions('node');
```

```

// Obtenir toutes les d finitions pour un bundle sp cifique
$bundle_fields = \Drupal::service('entity_field.manager')
  ->getFieldDefinitions('node', 'article');

// Obtenir la d finition d'un seul champ
$title_definition = $bundle_fields['title'];
$label     = $title_definition->getLabel();
$type      = $title_definition->getType();
$settings  = $title_definition->getSettings();

// Obtenir les d finitions depuis une entit  charg e
$node = \Drupal\node\Entity\Node::load(1);
$definitions = $node->getFieldDefinitions();
$body_def    = $node->getFieldDefinition('body');
```

Listing 1.7 – Récupérer les définitions de champs par programmation

1.5 Plusieurs formateurs de champ pour un même type de champ

Oui, il est tout à fait possible de créer plusieurs formateurs pour le même type de champ. Chaque formateur est un plugin séparé annoté avec `@FieldFormatter`.

```php
<?php
// Formateur 1 : Affichage en texte brut
namespace Drupal\mymodule\Plugin\Field\FieldFormatter;

use Drupal\Core\Field\FormatterBase;
use Drupal\Core\Field\FieldItemListInterface;

/**
 * @FieldFormatter(
 *   id = "mymodule_plain",
 *   label = @Translation("Mon texte brut"),
 *   field_types = {"string"},
 * )
 */
class MyPlainFormatter extends FormatterBase {
  public function viewElements(
      FieldItemListInterface $items, $langcode) {
    $elements = [];
    foreach ($items as $delta => $item) {
      $elements[$delta] = ['#markup' => $item->value];
    }
    return $elements;
  }
}

// Formateur 2 : Affichage en majuscules (m me type de champ
   !)
/**
```

```
 * @FieldFormatter(
 *   id = "mymodule_uppercase",
 *   label = @Translation("Mon majuscules"),
 *   field_types = {"string"},
 * )
 */
class MyUppercaseFormatter extends FormatterBase {
  public function viewElements(
      FieldItemListInterface $items, $langcode) {
    $elements = [];
    foreach ($items as $delta => $item) {
      $elements[$delta] = [
        '#markup' => strtoupper($item->value),
      ];
    }
    return $elements;
  }
}
```

Listing 1.8 – Deux formateurs pour le même type de champ

Les deux formateurs apparaissent comme options dans l'interface *Gérer l'affichage* pour tout champ utilisant le type `string`.

1.6 Récupérer la configuration d'un module via Drush

```
# Lister tous les objets de config d'un module
drush config:list | grep mymodule

# Obtenir un objet de configuration complet
drush config:get mymodule.settings

# Obtenir une cl   sp cifique dans un objet de config
drush config:get mymodule.settings some_key

# Exporter un objet de config en YAML vers stdout
drush config:get --format=yaml mymodule.settings

# Modifier une valeur de config directement
drush config:set mymodule.settings some_key "nouvelle_valeur"
```

Listing 1.9 – Commandes Drush pour inspecter la configuration

Jour 2: Travailler avec les Hooks

Références : Drupalize.me et Drupal At Your Fingertips

2.1 Ajouter un champ de base via `hook_entity_base_field_info`

```php
<?php

use Drupal\Core\Entity\EntityTypeInterface;
use Drupal\Core\Field\BaseFieldDefinition;

/**
 * Implements hook_entity_base_field_info().
 */
function mymodule_entity_base_field_info(
    EntityTypeInterface $entity_type) {
  $fields = [];

  if ($entity_type->id() === 'node') {
    $fields['my_custom_field'] =
       BaseFieldDefinition::create('string')
      ->setLabel(t('Mon champ personnalis '))
      ->setDescription(t('Un champ personnalis  sur tous les
         n uds.'))
      ->setRevisionable(TRUE)
      ->setTranslatable(TRUE)
      ->setDisplayOptions('view', [
        'label'  => 'above',
        'type'   => 'string',
        'weight' => 10,
      ])
      ->setDisplayOptions('form', [
        'type'   => 'string_textfield',
        'weight' => 10,
      ])
      ->setDisplayConfigurable('form', TRUE)
      ->setDisplayConfigurable('view', TRUE);
  }

  return $fields;
}
```

Listing 2.1 – Ajouter un champ de base personnalisé aux nœuds

Après avoir ajouté le hook, exécutez `drush entity:updates` (ou utilisez `hook_update_N`) pour appliquer les changements de schéma.

2.2 Rôle de hook_update_N

`hook_update_N` fournit un moyen structuré d'exécuter des mises à jour ponctuelles de base de données ou de configuration lors de la mise à jour d'un module. Le suffixe `N` est un entier combinant la version du schéma du module.

```php
<?php

/**
 * Ajouter le champ de base 'my_custom_field' aux
    enregistrements
 * de noeuds existants.
 */
function mymodule_update_9001(&$sandbox) {
  // Installer un nouveau sch ma de stockage de champ
  $field_storage = \Drupal::entityDefinitionUpdateManager()
    ->getFieldStorageDefinition('my_custom_field', 'node');

  if (!$field_storage) {
    $definition =
       \Drupal\Core\Field\BaseFieldDefinition::create('string')
      ->setLabel(t('Mon champ personnalis '));
    \Drupal::entityDefinitionUpdateManager()
      ->installFieldStorageDefinition(
        'my_custom_field', 'node', 'mymodule', $definition
      );
  }
}
```

Listing 2.2 – Exemple hook_update_N

- S'exécute une seule fois par site lors de l'exécution de `drush updatedb` (ou `update.php`).
- Ne s'exécute plus jamais après avoir été appliqué.
- Idéal pour : les changements de schéma, les migrations de données, les mises à jour de configuration.

2.3 Rôle de hook_install

`hook_install` se déclenche une seule fois lors de la première activation d'un module. C'est l'endroit approprié pour les tâches de configuration qui ne doivent se produire qu'au moment de l'installation.

```php
<?php

```

```
/**
 * Implements hook_install().
 */
function mymodule_install() {
  // Cr er un terme de taxonomie par d faut
  $term = \Drupal\taxonomy\Entity\Term::create([
    'vid'  => 'tags',
    'name' => 'Tag par d faut',
  ]);
  $term->save();

  // D finir une valeur de configuration par d faut
  \Drupal::configFactory()
    ->getEditable('mymodule.settings')
    ->set('enabled', TRUE)
    ->save();
}
```

Listing 2.3 – hook_install dans mymodule.install

2.4 Préfixer les nœuds Article avec `HEY-` via `hook_ENTITY_TYPE_presave`

```
<?php

use Drupal\node\NodeInterface;

/**
 * Implements hook_ENTITY_TYPE_presave() for node entities.
 */
function mymodule_node_presave(NodeInterface $node) {
  if ($node->bundle() === 'article' && $node->isNew()) {
    $current_title = $node->getTitle();
    if (strpos($current_title, 'HEY-') !== 0) {
      $node->setTitle('HEY-' . $current_title);
    }
  }
}
```

Listing 2.4 – Préfixer les nouveaux titres de nœuds article avec HEY-

La vérification `$node->isNew()` garantit que le préfixe n'est ajouté qu'une seule fois à la création, pas à chaque sauvegarde ultérieure.

2.5 Rôle de `$entity->original`

Lors de la mise à jour d'une entité (pas lors de la création), Drupal définit `$entity->original` sur l'objet entité tel qu'il existait *avant* la sauvegarde en cours. Cela permet de comparer les anciennes et nouvelles valeurs.

```
<?php

```

```
use Drupal\node\NodeInterface;

/**
 * Implements hook_node_presave().
 */
function mymodule_node_presave(NodeInterface $node) {
  if (!$node->isNew() && isset($node->original)) {
    $old_title = $node->original->getTitle();
    $new_title = $node->getTitle();

    if ($old_title !== $new_title) {
      \Drupal::logger('mymodule')->notice(
        'Titre du noeud @id change de "@old" en "@new".',
        ['@id' => $node->id(), '@old' => $old_title,
         '@new' => $new_title]
      );
    }
  }
}
```

Listing 2.5 – Utiliser $entity->original pour détecter les changements

2.6 Surcharger un hook de thème d'un autre module

Pour surcharger un hook de thème fourni par un autre module, implémentez `hook_theme_registry_alte` pour rediriger la fonction de thème, ou fournissez simplement votre propre fichier de template qui prend la priorité selon l'ordre de suggestions de templates de Drupal.

```
<?php

/**
 * Implements hook_theme_registry_alter().
 */
function mymodule_theme_registry_alter(&$theme_registry) {
  // Surcharger le hook de th me 'views_view'
  if (isset($theme_registry['views_view'])) {
    // Pointer vers notre propre r pertoire de templates
    $theme_registry['views_view']['path'] =
      \Drupal::service('extension.list.module')
        ->getPath('mymodule') . '/templates';
    // Notre template : mymodule/templates/views-view.html.twig
  }
}
```

Listing 2.6 – Surcharger un hook de thème via hook_theme_registry_alter

Alternativement, dans un thème, placez simplement un fichier `.html.twig` correspondant dans `templates/` et Drupal l'utilisera automatiquement.

2.7 Ajouter des suggestions de thème pour l'utilisateur selon le mode d'affichage

```php
<?php

/**
 * Implements hook_theme_suggestions_alter().
 */
function mymodule_theme_suggestions_alter(
    array &$suggestions, array $variables, $hook) {
  if ($hook === 'user') {
    // $variables['elements']['#view_mode'] contient le mode
       d'affichage
    if (isset($variables['elements']['#view_mode'])) {
      $view_mode = $variables['elements']['#view_mode'];
      // Ajoute la suggestion : user--{view_mode}.html.twig
      $suggestions[] = 'user__' . $view_mode;
      // Plus sp cifique : user--{uid}--{view_mode}.html.twig
      if (isset($variables['elements']['#user'])) {
        $uid = $variables['elements']['#user']->id();
        $suggestions[] = 'user__' . $uid . '__' . $view_mode;
      }
    }
  }
}
```

Listing 2.7 – Ajouter une suggestion de thème utilisateur par mode d'affichage

Drupal cherchera maintenant des templates tels que :

— `user-teaser.html.twig` lors du rendu en mode d'affichage *teaser*
— `user-full.html.twig` lors du rendu en mode d'affichage *full*

Jour 3: Travailler avec la Configuration & les Fonctionnalités

Références : Aperçus YouTube et Drupal At Your Fingertips

3.1 Configuration de l'environnement

Deux instances Drupal séparées ont été configurées :

- **Dev** — Environnement de développement (local)
- **Prod** — Environnement de production simulé (local)

3.2 Workflow de synchronisation de configuration

3.2.1 Dev : Configuration

```
# Installer Devel
composer require drupal/devel
drush en devel -y

# Cr er la taxonomie, le type de contenu, le r le via l'UI,
   puis exporter
drush config:export -y
```

Listing 3.1 – Installer Devel et configurer le contenu

3.2.2 Production : Import

```
# Copier le r pertoire config/sync de dev vers prod, puis :
drush config:import -y
drush cr
```

Listing 3.2 – Importer la configuration en production

Question

Devel a-t-il été installé en production ?

Oui — parce que le `config:export` standard inclut *tous* les modules activés. Sans intervention, `devel` est importé et activé en production, ce qui est indésirable.

Solution : le module `config_split`.

3.2.3 Utilisation de Config Split

`config_split` permet une configuration spécifique à l'environnement. Configurez-le pour que le module `devel` soit exclu des exports de production :

```
composer require drupal/config_split
drush en config_split -y

# Configurer    /admin/config/development/configuration/config-
   split
# Cr er un split "D veloppement" qui inclut le module devel
# D finir le r pertoire du split, puis exporter

drush config:export -y
# En production : importer sans la config devel
drush config:import -y
```

Listing 3.3 – Configuration de Config Split

3.3 Workflow du module Features

```
# Dev : Cr er et exporter une fonctionnalit
composer require drupal/features
drush en features features_ui -y

# Regrouper votre type de contenu + taxonomie + champs
# dans une fonctionnalit  via l'UI
# /admin/config/development/features, puis :
drush features:export myfeature -y

# Cela cr e un module dans modules/custom/myfeature/

# D ployer en production (copier le r pertoire du module),
   puis :
# La prod a features mais PAS features_ui :
drush en myfeature -y
drush features:import myfeature -y
drush cr
```

Listing 3.4 – Workflow Features : Dev vers Prod

Question

Y avait-il des conflits après l'ajout d'un nouveau champ ?

Oui. Après avoir renommé le type de contenu en production puis ajouté un nouveau champ en dev et réexporté la fonctionnalité, un conflit est apparu car le libellé du type de contenu différait entre les environnements. La résolution du conflit nécessitait soit :

— Revenir à la version de la fonctionnalité avec `drush features:import -force myfeature`, **ou**

— Réconcilier manuellement les valeurs de config divergentes avant l'import.

Jour 4: Travailler avec le Cache

Références : Drupalize.me et Drupal At Your Fingertips

4.1 Point de terminaison personnalisé /api/articles

4.1.1 Définition de la route

```
mymodule.api_articles:
  path: '/api/articles'
  defaults:
    _controller: '\Drupal\mymodule\Controller\
       ArticlesController::articles'
    _title: 'API Articles'
  requirements:
    _permission: 'access content'
```

Listing 4.1 – mymodule.routing.yml

4.1.2 Contrôleur avec mise en cache Max-Age

```
<?php

namespace Drupal\mymodule\Controller;

use Drupal\Core\Controller\ControllerBase;
use Symfony\Component\HttpFoundation\JsonResponse;
use Drupal\Core\Cache\CacheableJsonResponse;
use Drupal\Core\Cache\CacheableMetadata;

class ArticlesController extends ControllerBase {

  public function articles() {
    $nids = [10, 223, 45]; // IDs de noeuds cod s en dur

    $nodes = \Drupal::entityTypeManager()
      ->getStorage('node')
      ->loadMultiple($nids);

    $data = [];
    foreach ($nodes as $node) {
      $data[] = [
```

```
22          'nid'   => (int) $node->id(),
23          'title' => $node->getTitle(),
24        ];
25      }
26
27      // --- Approche Max-Age ---
28      $response = new CacheableJsonResponse($data);
29      $cache_metadata = new CacheableMetadata();
30      $cache_metadata->setCacheMaxAge(300); // 5 minutes
31      $response->addCacheableDependency($cache_metadata);
32
33      return $response;
34    }
35  }
```

Listing 4.2 – ArticlesController.php — Approche Max-Age

Note

Avec **max-age uniquement**, la modification du titre d'un nœud ne rafraîchit *pas* les données instantanément. Le point de terminaison sert la réponse en cache pendant jusqu'à 300 secondes, quels que soient les changements de contenu.

4.1.3 Contrôleur avec tags de cache (Recommandé)

```
1  <?php
2
3  public function articles() {
4    $nids = [10, 223, 45];
5    $nodes = \Drupal::entityTypeManager()
6      ->getStorage('node')
7      ->loadMultiple($nids);
8
9    $data = [];
10   $cache_tags = ['node_list'];  // Large : tout changement de
       noeud
11
12   foreach ($nodes as $node) {
13     $data[] = [
14       'nid'   => (int) $node->id(),
15       'title' => $node->getTitle(),
16     ];
17     // Granulaire : invalider uniquement lors du changement
18     // de ces noeuds sp cifiques
19     $cache_tags = array_merge(
20       $cache_tags,
21       $node->getCacheTags()  // ex. ['node:10', 'node:45']
22     );
23   }
24
25   $response = new CacheableJsonResponse($data);
26   $cache_metadata = new CacheableMetadata();
```

```
  $cache_metadata->setCacheTags($cache_tags);
  $response->addCacheableDependency($cache_metadata);

  return $response;
}
```

Listing 4.3 – ArticlesController.php — Approche Cache Tags

Note

Avec les **cache tags**, la modification du titre du nœud #10 invalide immédiatement le cache pour ce point de terminaison. La prochaine requête récupère des données fraîches depuis la base de données.

4.2 Inspecter les tags de cache via les en-têtes de réponse

Après avoir activé le débogage du cache de rendu dans `settings.local.php` :

```
<?php
$settings['cache']['bins']['render'] =
  'cache.backend.null'; // D sactiver le cache de rendu pour
     le dev

// Activer les en-t tes de tags de cache :
$settings['http.response.debug_cacheability_headers'] = TRUE;
```

Listing 4.4 – settings.local.php — Activer le débogage du cache

Inspecter les en-têtes dans un navigateur ou avec curl :

```
curl -I http://votre-site.local/api/articles

# Cherchez ces en-t tes dans la r ponse :
# X-Drupal-Cache-Tags: node:10 node:45 node:223 node_list
# X-Drupal-Cache-Contexts: languages:language_interface url.
   site
# X-Drupal-Cache-Max-Age: -1
```

Listing 4.5 – Inspecter les tags de cache dans les en-têtes de réponse

Jour 5: Travailler avec la Migration de Données

Références : Série de blog Evolving Web sur la migration Drupal 8

5.1 Rôle de `migration_lookup`

`migration_lookup` est un plugin de traitement qui associe un ID source d'une migration *précédemment exécutée* à l'ID de destination créé lors de cette migration. Il est essentiel pour maintenir l'intégrité référentielle lors de la migration d'entités liées.

```
process:
  field_category:
    plugin: migration_lookup
    migration: migrate_taxonomy_terms
    source: category_id
    # Si aucune correspondance n'est trouv e , ignorer ce champ
    # (pas de cr ation de stub)
    no_stub: true
```

Listing 5.1 – Utilisation de migration_lookup pour une référence taxonomique

Sans `migration_lookup`, les IDs d'entités liées seraient des IDs sources bruts sans signification dans la base de données de destination.

5.2 Importer depuis une source de base de données MySQL

Pour migrer depuis une base de données MySQL distante plutôt que des fichiers CSV, utilisez le module **migrate_plus** et configurez un plugin `source` de type `mysql` :

```
composer require drupal/migrate_plus drupal/migrate_tools
drush en migrate_plus migrate_tools -y
```

Listing 5.2 – Installer migrate_plus

```
id: migrate_articles_from_mysql
label: 'Migrer les articles depuis une base MySQL'
source:
  plugin: sql
  key: legacy_db          # D fini dans settings.php
  query: >
    SELECT id, title, body_text, created_at
```

```
    FROM articles
    WHERE status = 1

destination:
  plugin: 'entity:node'

process:
  title: title
  body/value: body_text
  body/format:
    plugin: default_value
    default_value: basic_html
```

Listing 5.3 – YAML de migration avec source MySQL

```php
<?php
$databases['legacy_db']['default'] = [
  'database' => 'legacy_database',
  'username' => 'db_user',
  'password' => 'db_pass',
  'host'     => '127.0.0.1',
  'port'     => '3306',
  'driver'   => 'mysql',
  'prefix'   => '',
];
```

Listing 5.4 – Connexion à la base de données dans settings.php

5.3 Annuler une migration

```
# Annuler une migration sp cifique (supprime les entit s de
    destination)
drush migrate:rollback migrate_articles_from_mysql

# Annuler plusieurs migrations
drush migrate:rollback migrate_articles_from_mysql,
    migrate_taxonomy_terms

# V rifier le statut actuel des migrations
drush migrate:status

# Relancer apr s annulation
drush migrate:import migrate_articles_from_mysql
```

Listing 5.5 – Annuler et relancer des migrations

Note

L'annulation ne fonctionne que si la table de carte d'ID de migration contient encore des enregistrements. Si la table de carte est vide, Drupal ne peut pas savoir quels éléments supprimer.

5.4 Traiter les données de champ avant l'import

La migration Drupal fournit un ensemble riche de **plugins de traitement** pour transformer les données. Pour tronquer une chaîne à 10 caractères :

```
process:
  title:
    plugin: callback
    callable: substr
    source: original_title
    # substr(string, start, length)
    # Arguments suppl mentaires pass s en param tres :
    unpack_source: false
```

Listing 5.6 – Utiliser le plugin callback pour tronquer une chaîne

Une approche plus propre consiste à utiliser un plugin de traitement personnalisé :

```
<?php

namespace Drupal\mymodule\Plugin\migrate\process;

use Drupal\migrate\ProcessPluginBase;
use Drupal\migrate\MigrateExecutableInterface;
use Drupal\migrate\Row;

/**
 * @MigrateProcessPlugin(
 *   id = "trim_string"
 * )
 */
class TrimString extends ProcessPluginBase {

  public function transform(
      $value,
      MigrateExecutableInterface $migrate_executable,
      Row $row,
      $destination_property) {
    $length = $this->configuration['length'] ?? 10;
    return substr($value, 0, $length);
  }
}
```

Listing 5.7 – Plugin de traitement personnalisé TrimString

```
process:
  title:
    plugin: trim_string
    source: original_title
    length: 10
```

Listing 5.8 – Utilisation du plugin trim_string personnalisé